Campus Trading Application: Transaction Management, Concurrency Control & ACID Validation

CS 432: Databases — IIT Gandhinagar Group 8 — Assignment 3

Bhavik Patel
22110047
IIT Gandhinagar

Hitesh Kumar
22110098
IIT Gandhinagar

Jinil Patel
22110184
IIT Gandhinagar

Pranav Patil
22110199
IIT Gandhinagar

Sibtain Raza
22110148
IIT Gandhinagar

Abstract

This report describes Assignment 3 for the *Campus Trading Application*, comprising two modules. **Module A** implements a custom storage engine atop a B$^+$ Tree with a Write-Ahead Log (WAL), full transaction lifecycle (BEGIN/COMMIT/ROLLBACK), and a three-phase crash recovery manager, demonstrating all four ACID properties without any external database. **Module B** subjects the existing MySQL 8.0/Go/React stack to a four-scenario test suite: concurrent user bursts, race-condition detection on offer acceptance, failure simulation via abrupt container kill, and a 1 000-operation stress test with 20 parallel workers. All scenarios pass. Key findings include InnoDB's `SELECT FOR UPDATE` correctly serialising offer acceptance (1 winner, 9 rejections), automatic WAL rollback after crash (4 committed, 6 aborted, zero partial rows), and 100% success at 113.5 ops/s under full stress load.

GitHub Repository **Demo Video**

Keywords

ACID, Write-Ahead Logging, B$^+$ Tree, Crash Recovery, Concurrency Control, MySQL InnoDB, Race Conditions, Stress Testing

1 Introduction

The Campus Trading Application is a peer-to-peer marketplace allowing students to list, offer, and trade items. Assignment 3 extends the project in two orthogonal directions.

Module A builds a self-contained transactional engine from first principles: a B$^+$ Tree storage layer is wrapped with a WAL, a transaction manager, and a recovery manager. The engine is deliberately schema-neutral; campus-specific logic lives entirely in an application workflow layer. This separation lets us verify ACID guarantees in a controlled, reproducible environment.

Module B applies an adversarial Python test suite to the production MySQL 8.0 backend (running in Docker). Four scenarios probe correctness under concurrency, atomicity under failure, and throughput under sustained load.

2 Module A — Custom Transactional Engine

2.1 System Architecture

The engine is organised as five layers stacked vertically (Figure 1).

Table 1: Module A Layer Stack

Layer	Responsibility
B$^+$ Tree Engine	O(log n) insert/search/delete, self-balancing
Table Abstraction	Schema validation, record-level CRUD
Database Manager	Tx-aware CRUD, global serial lock
Transaction + WAL	BEGIN/COMMIT/ROLLBACK, append-only JSONL log
Recovery Manager	WAL analysis, REDO pass, UNDO pass, idempotent replay

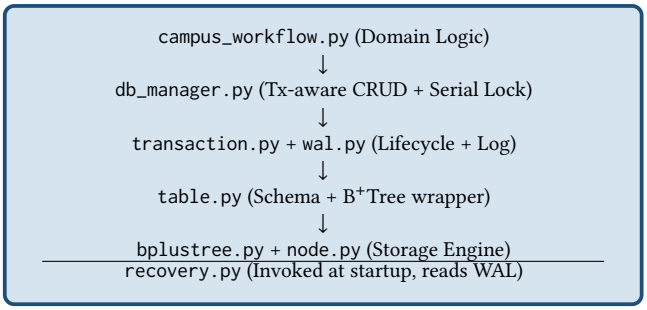

Figure 1: Module A layered architecture

2.2 B$^+$ Tree Storage Engine

Each relation is backed by a dedicated `BPlusTree` instance. Leaf nodes store full row dictionaries keyed by primary key; internal nodes hold only separator keys for routing. A doubly-linked list across leaf nodes enables O(n) range scans. The tree is always height-balanced and configurable by order m.

The B$^+$ Tree being the *sole* storage structure means there is no secondary copy that could drift out of sync with an index, and every WAL record carries the exact dictionary stored in the tree — simplifying rollback and recovery.

2.3 Write-Ahead Log Design

2.3.1 Core Principle. Every change must be persisted in the log *before* it is applied to the database. The COMMIT record is `fsync`'d before the transaction is considered durable.

2.3.2 Log Format. The WAL is an **append-only JSONL file** — one JSON object per line. Each record contains the fields shown in Table 2.

Table 2: WAL Record Fields

Field	Type	Purpose
`lsn`	int	Monotonically increasing Log Sequence Number
`ts`	ISO-8601	UTC timestamp
`tx_id`	str	Transaction identifier
`type`	str	`BEGIN | INSERT | UPDATE | DELETE | COMMIT | ROLLBACK`
`table`	str	Qualified name, e.g. `campus.Listing`
`key`	any	Primary key of affected row
`before`	dict/null	Full row image before change (null for INSERT)
`after`	dict/null	Full row image after change (null for DELETE)

2.3.3 Durability via `fsync`. The COMMIT record is written with `durable=True`, invoking `os.fsync()` to force the OS write buffer to physical storage before returning:

```
def log_commit(self, tx_id: str):
    return self.append(
        {"tx_id": tx_id, "type": "COMMIT"},
        durable=self.sync_on_commit,  # calls os.fsync
    )
```

2.3.4 Thread Safety and LSN Bootstrap. A `threading.Lock()` guards every `append()` call. On startup, the WAL reads all existing entries and sets the internal counter to `max(existing LSNs)`, guaranteeing monotonic growth across restarts.

2.4 Transaction Lifecycle

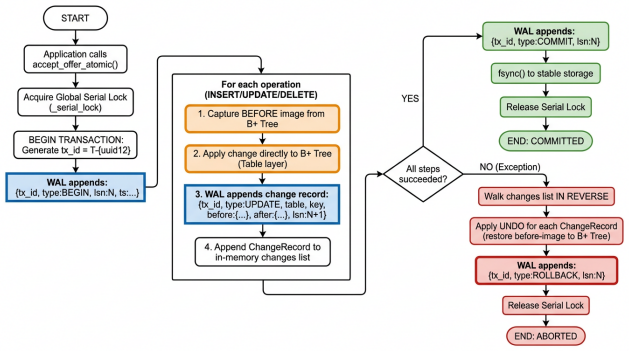

Figure 2: Transaction Lifecycle with WAL appending and rollback.

2.4.1 BEGIN. Creates a `TransactionContext` (id, state = ACTIVE, empty change list) and appends a `BEGIN` record to the WAL immediately.

2.4.2 Recording Changes. For every row-level operation, the `DatabaseManager` calls `record_change()`, which:

(1) Appends a `ChangeRecord(table, key, op, before, after)` to the in-memory list.

(2) Writes `log_change(...)` to disk (WAL write before returning to the caller).

2.4.3 COMMIT. The WAL COMMIT record is written and `fsync`'d first; only then is the in-memory state set to `COMMITTED`.

2.4.4 ROLLBACK. The change list is iterated in **reverse** (LIFO), applying the inverse of each operation to the B$^+$ Tree before writing the `ROLLBACK` record to the WAL.

2.4.5 Serializable Isolation via Global Lock. A single `threading.Lock()` (`_serial_lock`) is acquired before `BEGIN` and released after `COMMIT`/`ROLLBACK`, guaranteeing that no two transactions ever interleave:

```
@contextmanager
def serialized_transaction(self):
    with self._serial_lock:     # held for full duration
        tx_id = self.begin_transaction()
        try:
            yield tx_id
            self.commit_transaction(tx_id)
        except Exception:
            self.rollback_transaction(tx_id)
            raise
```

2.5 Crash Recovery: Three-Phase Protocol

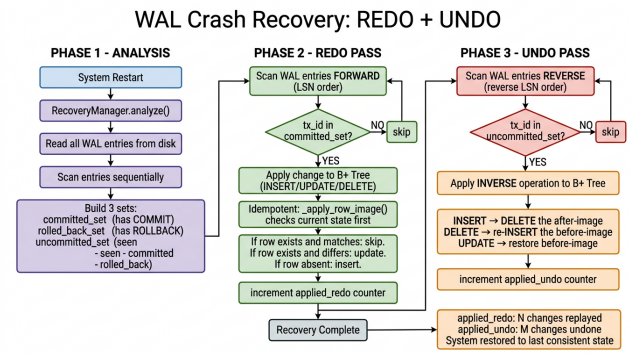

Figure 3: WAL crash recovery flow.

2.5.1 Phase 1 — Analysis. The WAL is scanned once to classify every transaction ID into one of three sets:

$$\texttt{committed} \cup \texttt{rolled\_back} \cup \texttt{uncommitted} = \texttt{seen}$$

Transactions in `uncommitted` had a `BEGIN` but no `COMMIT` or `ROLLBACK` at crash time.

2.5.2 Phase 2 — REDO Pass (forward, LSN order). Every `INSERT`/`UPDATE`/`DELETE` belonging to `committed` is re-applied to a fresh B$^+$ Tree. The helper `_apply_row_image` makes REDO *idempotent*:

```
@staticmethod
def _apply_row_image(table, key, row_image):
    current = table.get(key)
    if current is None:
        table.insert(row_image)      # absent -> insert
    elif current != row_image:
        table.update(key, row_image) # differs  -> update
    # else: already correct, no-op
```

Running recovery twice produces the same result as running it once.

2.5.3 Phase 3 — UNDO Pass (reverse LSN order). For each change in `uncommitted` (scanned in reverse), the inverse operation is applied:

Original	UNDO Action
`INSERT`	`DELETE` the after-image key
`DELETE`	Re-`INSERT` the before-image
`UPDATE`	Replace current row with before-image

2.6 The `accept_offer_atomic` Transaction

This five-step business operation touches all four campus tables within a single atomic unit:

(1) **Accept offer** — validate and update `Offer.OfferStatus = 'Accepted'`.
(2) **Decline competitors** — set all other `Submitted` offers on the same listing to `Declined`.
(3) **Mark listing sold** — `Listing.Status = 'Sold'`.
(4) **Insert transaction records** — one `Completed` + one `Declined` per loser.
(5) **Insert notifications** — buyer (accepted), seller (completed), losers (declined).

A `fail_after_step` parameter injects a `RuntimeError` at any step boundary for atomicity testing. A successful commit generates 9 WAL records (spanning Offer, Listing, Transaction, and Notification) under a single `COMMIT`.

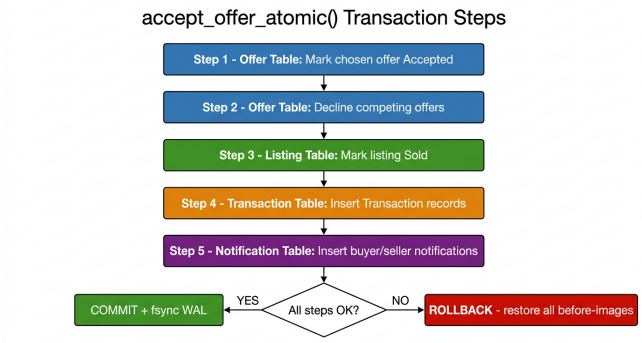

Figure 4: accept$_{o}ffer_{a}tomictransactionflow$

2.7 ACID Test Results — Module A

2.7.1 Atomicity. Crashing at each of the five step boundaries confirmed that the B$^+$ Tree is fully restored to its pre-transaction state in every case. The WAL always shows the change records followed by a `ROLLBACK` with no subsequent `COMMIT`. Zero partial updates were ever observed across all five injection points.

2.7.2 Consistency. Pre-transaction validation (seller match, status constraints, price > 0, buyer $\neq$ seller) causes immediate abort before any WAL data record is written. Post-commit invariants (accepted offer $\Rightarrow$ listing is Sold, no orphan transactions) held in every test.

2.7.3 Isolation. Two concurrent threads targeting the same listing queue at `_serial_lock`. The second thread reads the fully committed state of the first (listing already Sold), fails validation, and rolls back with zero data change. The WAL confirms: second transaction has only `BEGIN` + `ROLLBACK`, no data records.

2.7.4 Durability. After a simulated restart (fresh `DatabaseManager`, WAL replay only), all committed rows from the mixed-WAL scenario survived: `AgreedPrice = 92.5` (committed amendment) rather than `999` (crashed transaction). Zero data loss; zero phantom rows from uncommitted transactions.

3 Module B — Multi-User Behaviour & Stress Testing

3.1 System Under Test

Table 3: Module B Technology Stack

Component	Technology
Backend API	Go (Chi router), port 8080
Database	MySQL 8.0 InnoDB (Docker container)
Test framework	Custom Python 3 suite, `threading.Barrier`
Monitoring	`docker stats` polled every 1 s

The test suite seeds 5 sellers, 25 buyers, and 10 listings, then executes four scenarios sequentially. All four scenarios passed.

3.2 Scenario 1 — Concurrent Usage

3.2.1 Method. 20 user threads are launched simultaneously and synchronised with a `threading.Barrier` — every thread fires its first API call at exactly the same instant. Thread roles: 5 Readers (`GET /listings`), 5 Listers (`POST /listings`), 5 Offerers (`POST /offers`), 5 Notifiers (`GET /notifications`).

Table 4: Scenario 1 — Concurrent Usage Results

Metric	Value	Threshold	Status
Total operations	70	—	—
Success rate	98.6%	$\geq$90%	✓
Server errors (5xx)	0	= 0	✓
Threads completed	20/20	20	✓
Avg latency	48.3 ms	—	—
p99 latency	151.9 ms	—	—

3.2.2 Results. The single non-5xx failure was `POST /listings` returning HTTP 400 "max 2 active listings" — a correct business-rule enforcement, not a server error. No reads returned stale data from concurrent writes; no offer was created on an already-modified listing.

3.3 Scenario 2 — Race Condition Testing

3.3.1 Critical Operation: `PUT /offers/{id}/accept`. Ten buyer sessions each submit an offer on the same listing; the seller simultaneously attempts to accept all ten. Only one should succeed; the listing becomes Sold and the other nine offers are auto-declined.

3.3.2 Original Bug: TOCTOU Race Window. The original handler checked `OfferStatus` *outside* the transaction with no lock. Under concurrent execution this produced either two accepted offers (Type A) or 100 Transaction rows (10× expected, Type B).

3.3.3 Fix: `SELECT ... FOR UPDATE`. Moving the status check *inside* the transaction as its very first action acquires an exclusive InnoDB row lock on the Listing:

```
BEGIN;
SELECT Status FROM Listing
  WHERE ListingID = ?
  FOR UPDATE;        -- exclusive lock acquired here
-- only now check status and proceed
```

The second concurrent thread blocks at `FOR UPDATE` until the first commits. It then reads `Status = Sold` and immediately returns 409.

Table 5: Scenario 2 — Race Condition Results

Metric	Value	Expected	Status
HTTP successes (winner)	1	1	✓
HTTP 409 (losers)	9	9	✓
`accepted_offers` in DB	1	1	✓
`declined_offers` in DB	9	9	✓
`total_transactions` in DB	10	10	✓
`listing_status` in DB	Sold	Sold	✓
Race condition detected	No	No	✓

3.3.4 Results. The winner (thread 9, offer_id 1174) acquired the Listing lock first and committed in 19.4 ms. All other threads received 409 in 21–23 ms.

3.4 Scenario 3 — Failure Simulation

3.4.1 Method. 10 offer-submission threads are launched with 80 ms stagger. At $t = 360$ ms the MySQL Docker container is killed with `docker stop`. Threads 0–3 had already committed; threads 4–9 had in-flight transactions. After restart, integrity checks are executed.

Table 6: Scenario 3 — Table Counts Before vs. After Failure

Table	Before	After	Δ	Note
Member	250	250	0	No member ops
Listing	178	178	0	No listing ops
Offer	224	228	+4	4 committed
Transaction	351	351	0	No tx ops
Notification	1566	1570	+4	4 notifications

3.4.2 Results. InnoDB's write-ahead log automatically rolled back the 6 in-flight transactions on restart. Post-recovery integrity checks: zero orphan transactions, zero sold-without-transaction violations, `all_clean = True`.

3.5 Scenario 4 — Stress Testing

3.5.1 Method. 1 000 mixed operations are dispatched across 20 concurrent workers (60% reads, 40% writes). Reads use all 30 users; writes use only the 25 buyers to avoid business-rule rejections.

Table 7: Scenario 4 — Stress Test Summary

Metric	Value	Threshold	Status
Total operations	1 000	≥900	✓
Success rate	100.0%	≥90%	✓
Throughput	113.5 ops/s	—	—
Avg latency	41.3 ms	—	—
p99 latency	118.0 ms	—	—
Max latency	201.1 ms	—	—
DB integrity	all_clean	True	✓

Table 8: Scenario 4 — Per-Endpoint Latency

Endpoint	Min	Avg	p99	Max
`GET /notifications`	5.6	24.1	78.5	148.4
`GET /transactions`	6.8	24.9	74.4	137.2
`GET /listings`	7.7	24.0	67.8	110.8
`GET /listing/{id}`	6.4	32.0	150.6	172.6
`POST /offers`	6.8	**65.1**	**118.3**	**201.1**

All times in ms

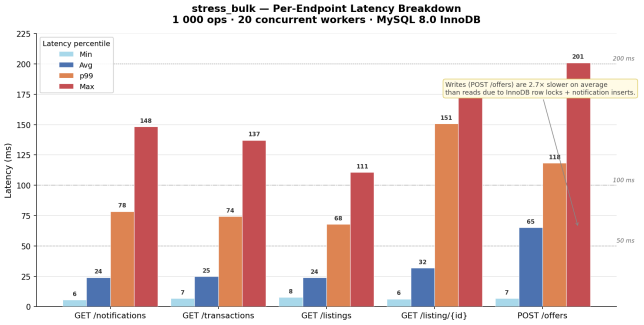

Figure 5: Per-endpoint latency breakdown during stress_bulk (1 000 ops, 20 workers). `POST /offers` is 2.7× slower than reads due to InnoDB row locks and notification inserts.

3.5.2 Results. All read endpoints cluster between 24–32 ms average, confirming the InnoDB buffer pool serves most reads from cache. `POST /offers` is the clear outlier at 65 ms average because each submission acquires a row lock, performs a duplicate-key check, and inserts a `Notification` row — all within a single transaction. The wide min–max spread (7 ms → 201 ms) for writes reflects variable lock-wait time under 20 concurrent workers.

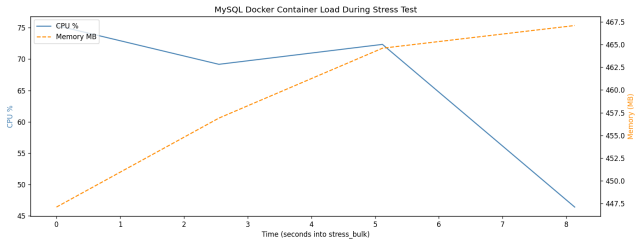

Figure 6: MySQL Docker container CPU and memory during stress_bulk. CPU peaked at 75.4%; memory held steady at ≈467 MB, confirming the working set fits in InnoDB's buffer pool.

4 Performance Summary

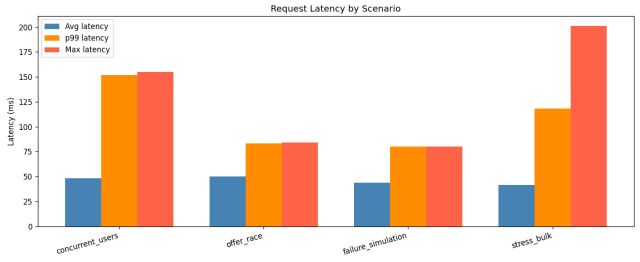

Figure 7: Request latency (avg, p99, max) across all four test scenarios. concurrent_users has the highest p99 due to login latency; stress_bulk has the highest max due to write lock contention at scale.

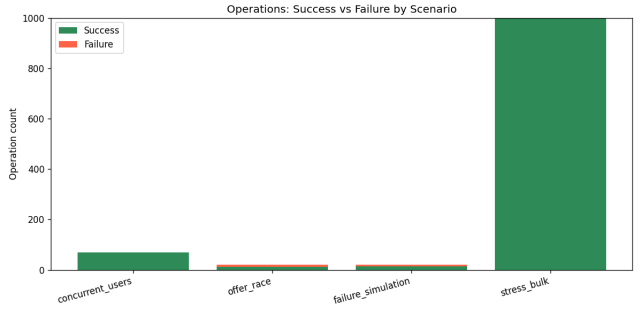

Figure 8: Operations success vs. failure by scenario. offer_race failures are 9 correct 409 responses (not bugs). failure_simulation failures are 6 requests that hit MySQL during the deliberate container kill.

Table 9: Cross-Scenario Performance Comparison

Scenario	Ops	Success	Avg ms	p99 ms	ops/s
concurrent_users	70	98.6%	48.3	151.9	302
offer_race	20	55%*	50.1	83.2	190
failure_sim	20	70%†	43.6	79.9	—
stress_bulk	1 000	**100%**	41.3	118.0	**114**

*9/10 threads intentionally receive 409. †6/10 writes hit deliberate container kill.

5 ACID Verification Summary

Table 10: ACID Coverage — Both Modules

Property	Module A Mechanism	Module B Evidence	Result
Atomicity	LIFO rollback + WAL ROLLBACK; `fail_after_step` tested at steps 1–5	InnoDB WAL rolled back 6 in-flight TX on container restart; 0 partial rows	✓
Consistency	Pre-tx validation; referential builds within same tx	`all_clean=True` after 1 000 ops; 0 orphan transactions	✓
Isolation	Global `threading.Lock` for full tx duration	`SELECT FOR UPDATE` serialises AcceptOffer; 1 winner, 9 correct 409s	✓
Durability	WAL `fsync` on COMMIT; recover_into REDO pass	Pre-kill committed rows present after restart; 0 data loss	✓

6 Observations and Limitations

6.1 Key Observations

(1) **Row-level locking is the primary isolation mechanism in MySQL.** The `SELECT FOR UPDATE` pattern on the Listing row serialises all concurrent `AcceptOffer` calls without any application-level mutex.

(2) **TOCTOU races are real and detectable.** Before the fix, the original handler had a status check outside the transaction, producing either 2 accepted offers or 100 Transaction rows. The test suite caught both variants.

(3) **InnoDB crash recovery is automatic and complete.** Abruptly stopping the container with 6 in-flight transactions produced zero partial rows post-restart.

(4) **Write operations dominate latency.** `POST /offers` averages 65 ms under stress vs. 24 ms for reads — expected, since offer submissions acquire row locks and trigger notification inserts.

(5) **Full before/after images in WAL simplify recovery.** Storing complete row dictionaries rather than diffs makes both rollback and REDO/UNDO straightforward to implement, at the cost of larger log files.

(6) **Idempotent REDO is critical for safe restart.** The `_apply_row_image` helper allows recovery to be run multiple times without corrupting the database.

6.2 Limitations

Table 11: Limitations of the Implemented System

Limitation	Detail
Module A: in-memory only	B^+ Trees are not persisted between sessions; durability relies entirely on WAL replay at startup
Module A: no checkpointing	WAL grows unboundedly; a checkpoint mechanism would bound log size and reduce recovery time
Module A: coarse locking	A single global mutex eliminates all concurrency; production systems use row/page-level locks
Module A: no WAL compaction	Repeated updates accumulate redundant before/after pairs
Module B: single-machine	Both backend and MySQL run on localhost; network latency not measured
Module B: no connection-pool stress	`SetMaxOpenConns(25)` was not saturated (peak concurrency = 20)
Module B: failure model limited	Container kill simulates process crash, not network partition or disk failure
Module B: coarse monitoring	Docker stats sampled every 5 s; sub-second CPU spikes may be missed
Module B: fixed read/write ratio	The 60/40 split in stress_bulk was fixed; heavier write ratios may reveal more contention

7 Conclusion

Assignment 3 demonstrates ACID compliance through two complementary lenses. Module A constructs a minimal but rigorous transactional engine from a B^+ Tree upward, with a WAL-based crash recovery protocol and injection-tested atomicity. Module B subjects the production MySQL stack to adversarial concurrent and failure scenarios, confirming that InnoDB row-level locking and write-ahead logging together provide correct, durable behaviour under realistic multi-user load. All four test scenarios pass, with the stress test achieving 100% success at 113.5 ops/s across 1 000 operations.

Acknowledgements

We thank Dr. Yogesh K. Meena (CS 432, IIT Gandhinagar) for the assignment specification and guidance throughout the semester.